

Lecture 19

Introduction to Dynamic Programming

Source: Introduction to Algorithms, CLRS

Introduction to Dynamic Programming

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems,

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems, when subproblems share subsubproblem.

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems, when subproblems share subsubproblem.

Idea:

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems, when subproblems share subsubproblem.

Idea: Instead of solving a subproblem repeatedly,

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems, when subproblems share subsubproblem.

Idea: Instead of solving a subproblem repeatedly, solve it once and store the result in an array.

Introduction to Dynamic Programming

Dynamic programming solves problem by combining the solutions to subproblems, when subproblems share subsubproblem.

Idea: Instead of solving a subproblem repeatedly, solve it once and store the result in an array.

Let's learn DP through an example!

Rod Cutting

Rod Cutting

Rod-Cutting:

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example:

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$,

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod:

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod:

Profit earned:

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod: (4)

Profit earned: 9

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod: (4) (1,3)

Profit earned: 9 9

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod: (4) (1,3) (2,2)

Profit earned: 9 9 10

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod: (4) (1,3) (2,2) (1,1,2)

Profit earned: 9 9 10 7

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod:	(4)	(1,3)	(2,2)	(1,1,2)	(1,1,1,1)
Profit earned:	9	9	10	7	4

Rod Cutting

Rod-Cutting:

Input: A rod of length n cm and an array $p[1 : n]$, where $p[i] \geq 0$ is the price of a rod of i cm long rod.

Output: Maximum profit obtainable by cutting (or not) the rod and selling the pieces.

Example: $n = 4$, $p =$

1	5	8	9
---	---	---	---

Possible cuttings for a length 4 rod: (4) (1,3) (2,2) (1,1,2) (1,1,1,1)

Profit earned: 9 9 10 7 4



Maximum profit

Brute Force Solution

Brute Force Solution

Brute force approach:

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.
- Output the **maximum profit** after calculating profit for all the cuttings.

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.
- Output the **maximum profit** after calculating profit for all the cuttings.

Time Complexity:

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.
- Output the **maximum profit** after calculating profit for all the cuttings.

Time Complexity: You can either cut or not cut at every i th inch.

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.
- Output the **maximum profit** after calculating profit for all the cuttings.

Time Complexity: You can either cut or not cut at every i th inch. Generating all cuttings this

Brute Force Solution

Brute force approach:

- Find all possible cuttings of the **length n rod**.
- Output the **maximum profit** after calculating profit for all the cuttings.

Time Complexity: You can either cut or not cut at every i th inch. Generating all cuttings this way can lead to **$O(2^n)$** time.

Towards a Better Algorithm

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is i , then $\textit{profit}_n = p[i] + \textit{profit}_{n-i}$

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

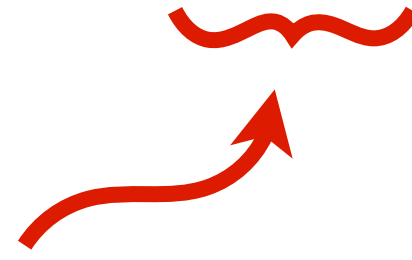
If length of the first cut in optimal cutting is i , then $\textit{profit}_n = \underbrace{p[i]} + \textit{profit}_{n-i}$

Towards a Better Algorithm

Let $profit_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

Price of the first cut

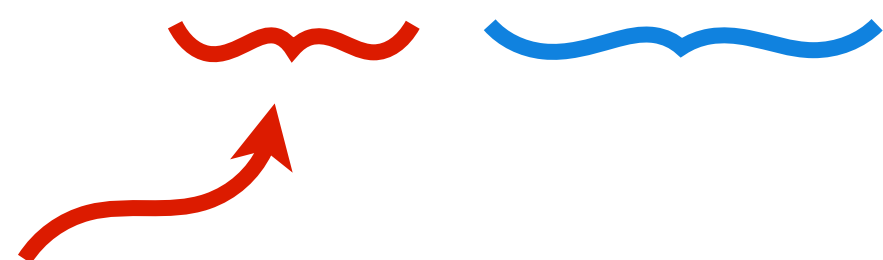


Towards a Better Algorithm

Let $profit_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

Price of the first cut



Towards a Better Algorithm

Let $profit_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

Price of the first cut

For the remaining $(n - i)$ length rod,
we cannot get more than $profit_{n-i}$.

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $\textit{profit}_n = p[1] + \textit{profit}_{n-1}$

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $\textit{profit}_n = p[1] + \textit{profit}_{n-1}$

If length of the first cut in optimal cutting is 2, then $\textit{profit}_n = p[2] + \textit{profit}_{n-2}$

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $\textit{profit}_n = p[1] + \textit{profit}_{n-1}$

If length of the first cut in optimal cutting is 2, then $\textit{profit}_n = p[2] + \textit{profit}_{n-2}$

If length of the first cut in optimal cutting is 3, then $\textit{profit}_n = p[3] + \textit{profit}_{n-3}$

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $\textit{profit}_n = p[1] + \textit{profit}_{n-1}$

If length of the first cut in optimal cutting is 2, then $\textit{profit}_n = p[2] + \textit{profit}_{n-2}$

If length of the first cut in optimal cutting is 3, then $\textit{profit}_n = p[3] + \textit{profit}_{n-3}$

⋮

Towards a Better Algorithm

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $\textit{profit}_n = p[1] + \textit{profit}_{n-1}$

If length of the first cut in optimal cutting is 2, then $\textit{profit}_n = p[2] + \textit{profit}_{n-2}$

If length of the first cut in optimal cutting is 3, then $\textit{profit}_n = p[3] + \textit{profit}_{n-3}$

⋮

If length of the first cut in optimal cutting is n (i.e. no cut is made), then $\textit{profit}_n = p[n]$

Towards a Better Algorithm

Let $profit_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $profit_n = p[1] + profit_{n-1}$

If length of the first cut in optimal cutting is 2, then $profit_n = p[2] + profit_{n-2}$

If length of the first cut in optimal cutting is 3, then $profit_n = p[3] + profit_{n-3}$

⋮

If length of the first cut in optimal cutting is n (i.e. no cut is made), then $profit_n = p[n]$

Towards a Better Algorithm

Let $profit_n$ = maximum profit obtainable from an n cm rod. Then,

If length of the first cut in optimal cutting is 1, then $profit_n = p[1] + profit_{n-1}$

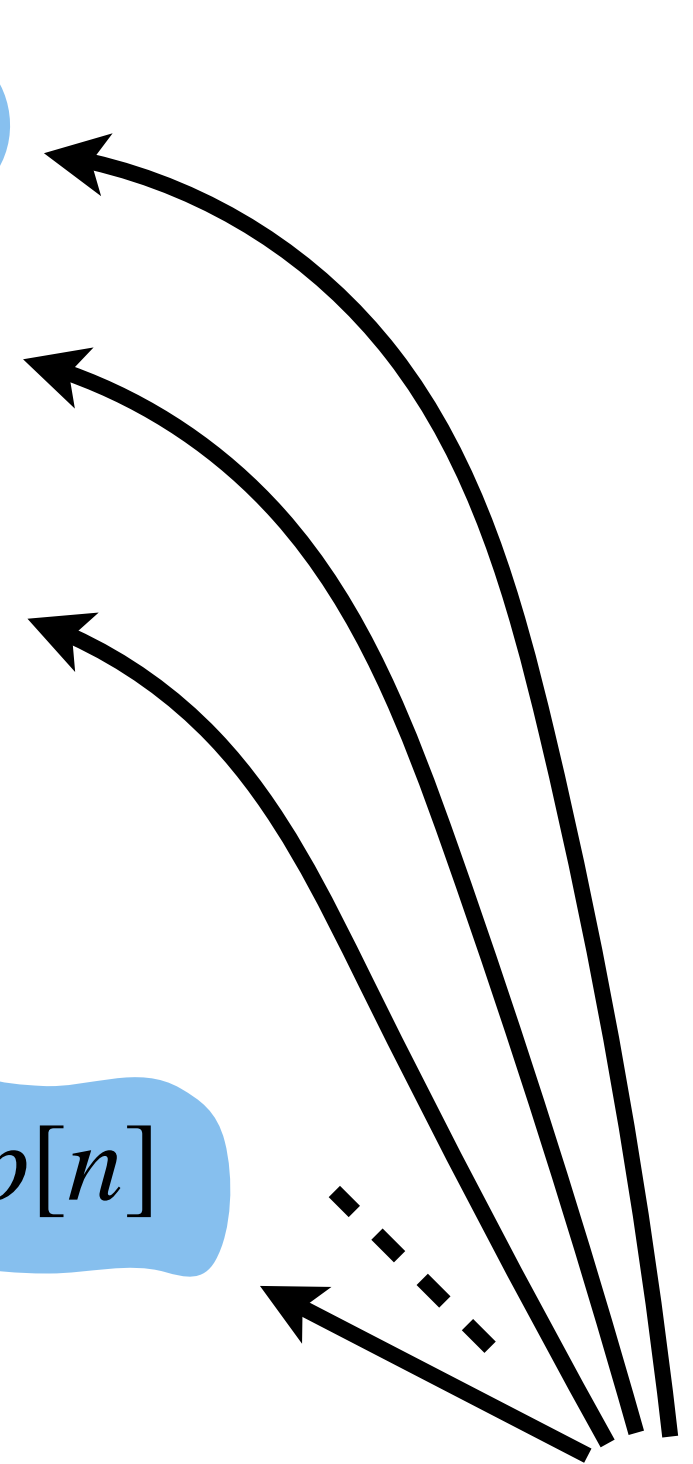
If length of the first cut in optimal cutting is 2, then $profit_n = p[2] + profit_{n-2}$

If length of the first cut in optimal cutting is 3, then $profit_n = p[3] + profit_{n-3}$

⋮

If length of the first cut in optimal cutting is n (i.e. no cut is made), then $profit_n = p[n]$

$profit_n$ is maximum of these



A Recursive Solution

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$
2. **return** $p[1]$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$
2. **return** $p[1]$
3. $\textit{profit} = p[n]$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$
2. **return** $p[1]$
3. $\textit{profit} = p[n]$
4. **for** $i = 1$ **to** $n - 1$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$
2. **return** $p[1]$
3. $\textit{profit} = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\textit{profit} = \text{Max}(\textit{profit}, p[i] + \text{RC}(n - i, p))$

A Recursive Solution

Let $\textit{profit}_n$ = maximum profit obtainable from an n cm rod. Then,

- $\textit{profit}_n = p[1]$, if $n = 1$
- $\textit{profit}_n = \max(p[n], \max_{i \in [1, n-1]}(p[i] + \textit{profit}_{n-i}))$, if $n > 1$

RC(n, p):

1. **if** $n == 1$
2. **return** $p[1]$
3. $\textit{profit} = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\textit{profit} = \text{Max}(\textit{profit}, p[i] + \text{RC}(n - i, p))$
6. **return** $\textit{profit}$

All Good?

All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time.

All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

Recursive tree of RC:

All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

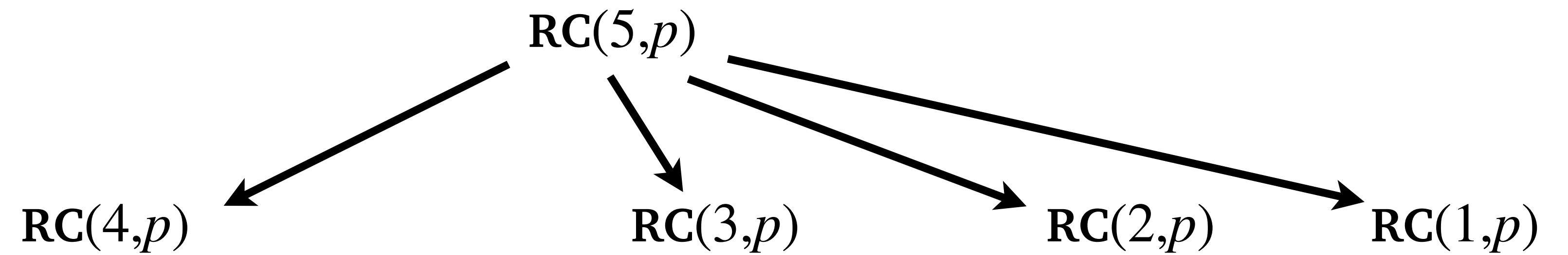
Recursive tree of RC:

$\text{RC}(5, p)$

All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

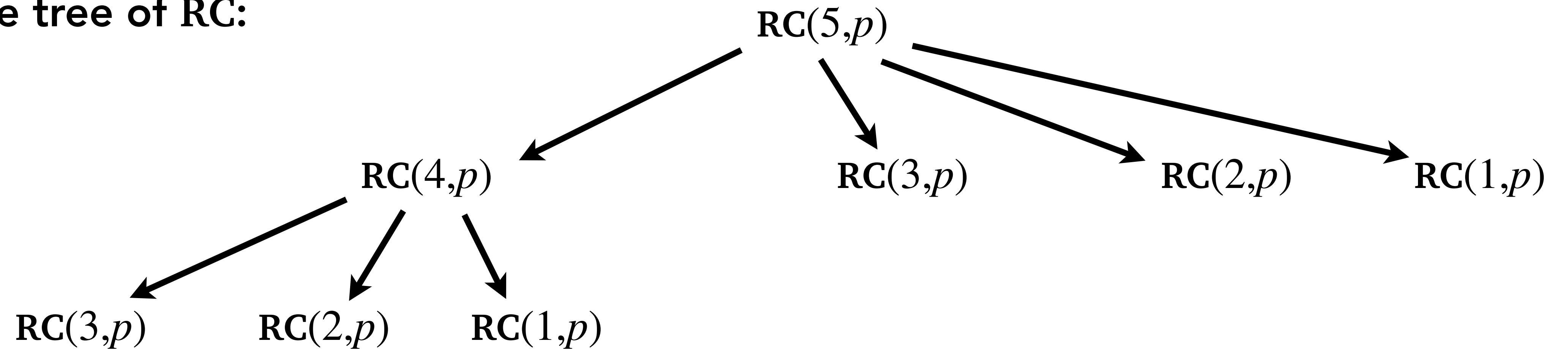
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

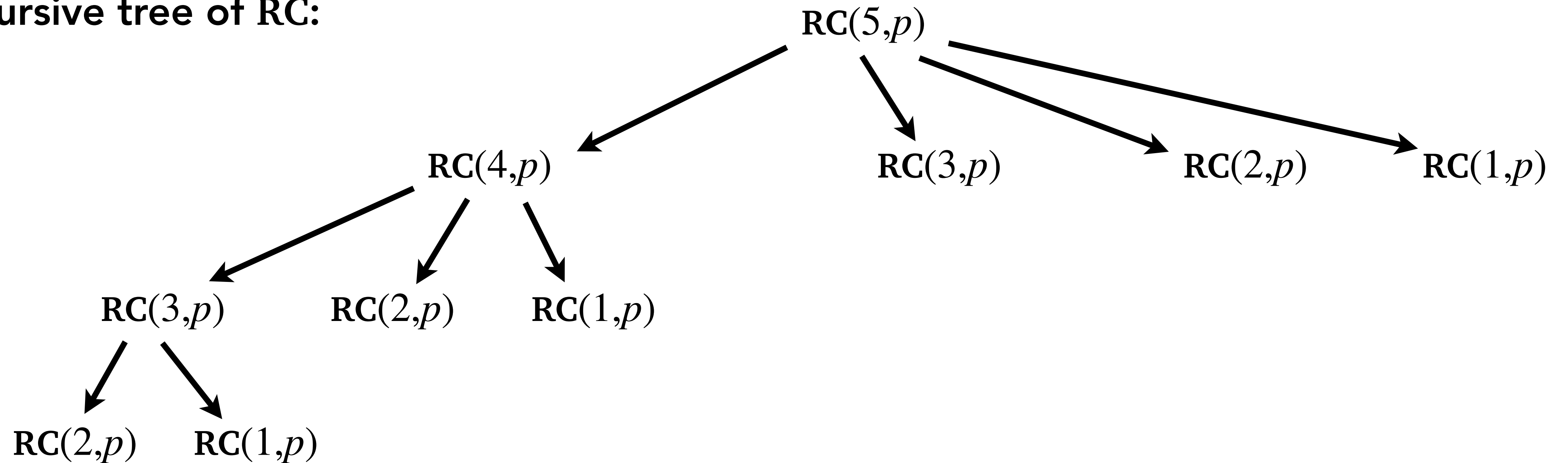
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

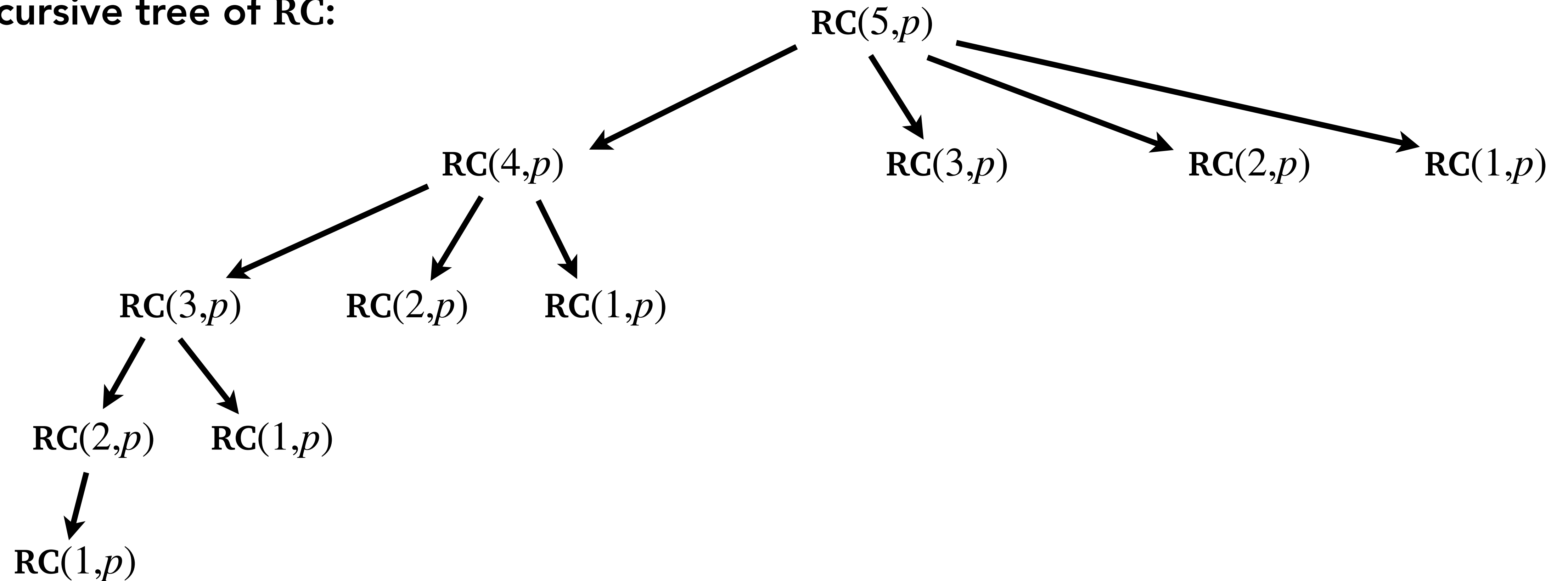
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

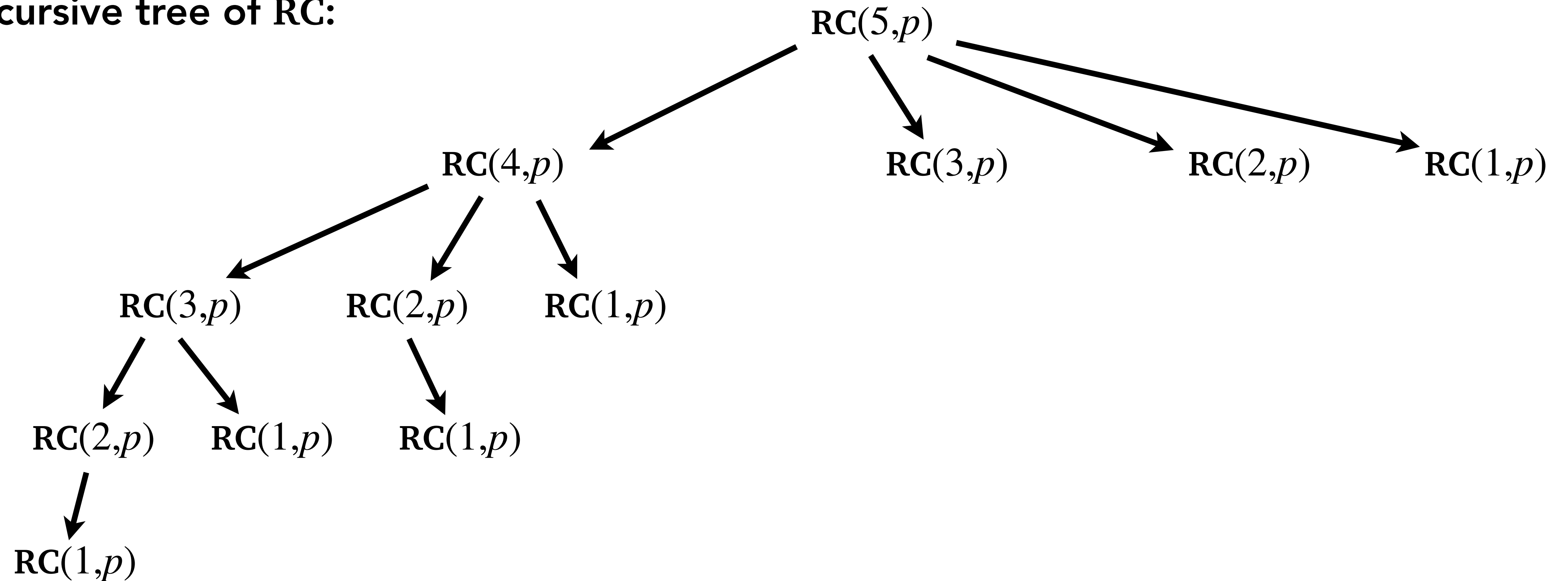
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

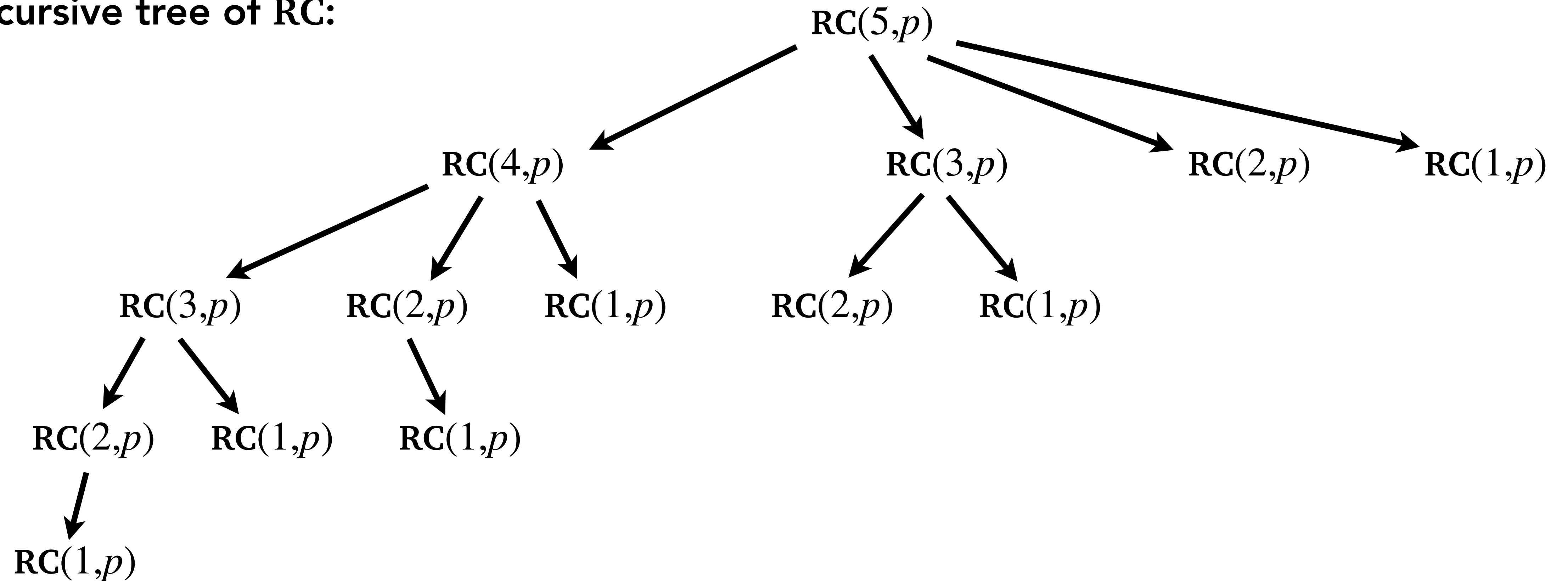
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

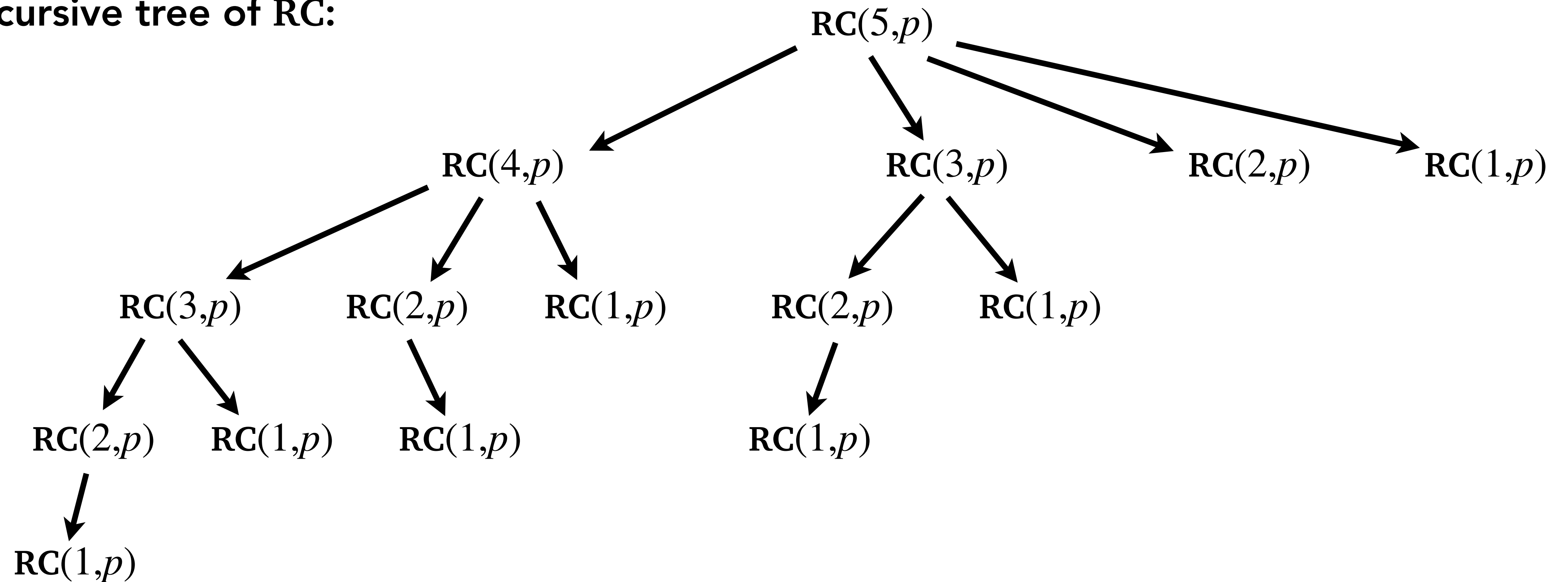
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

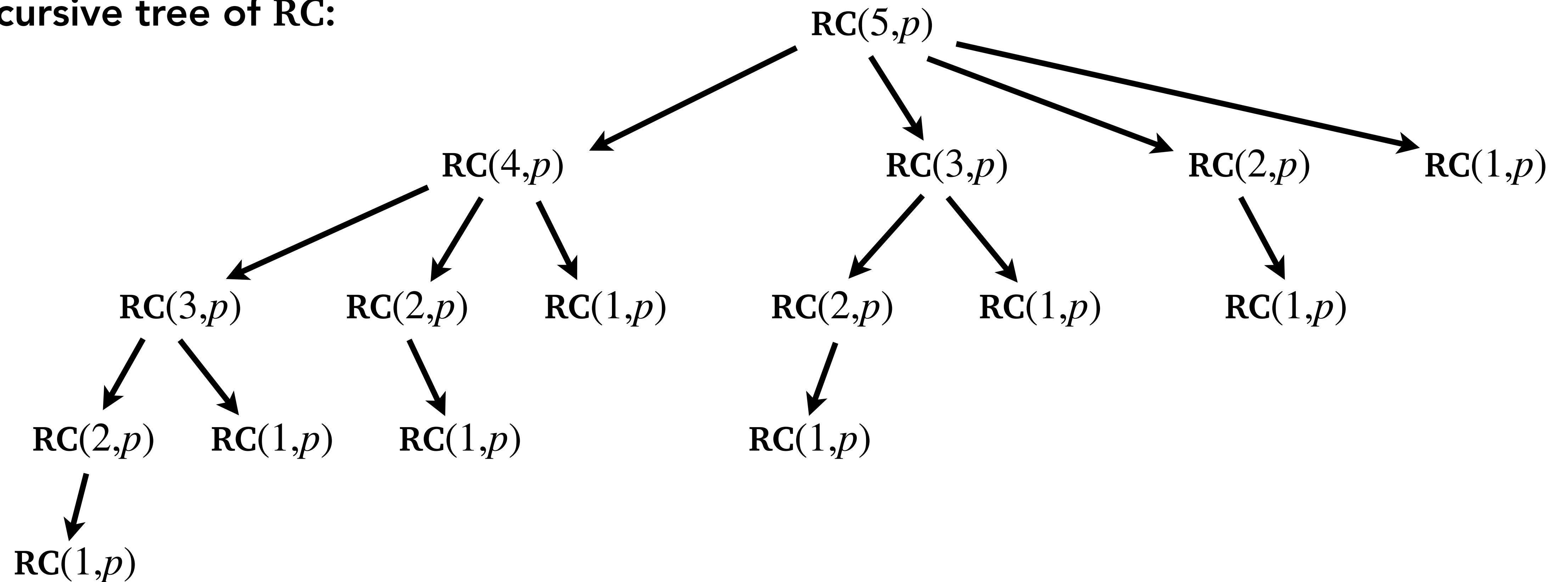
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

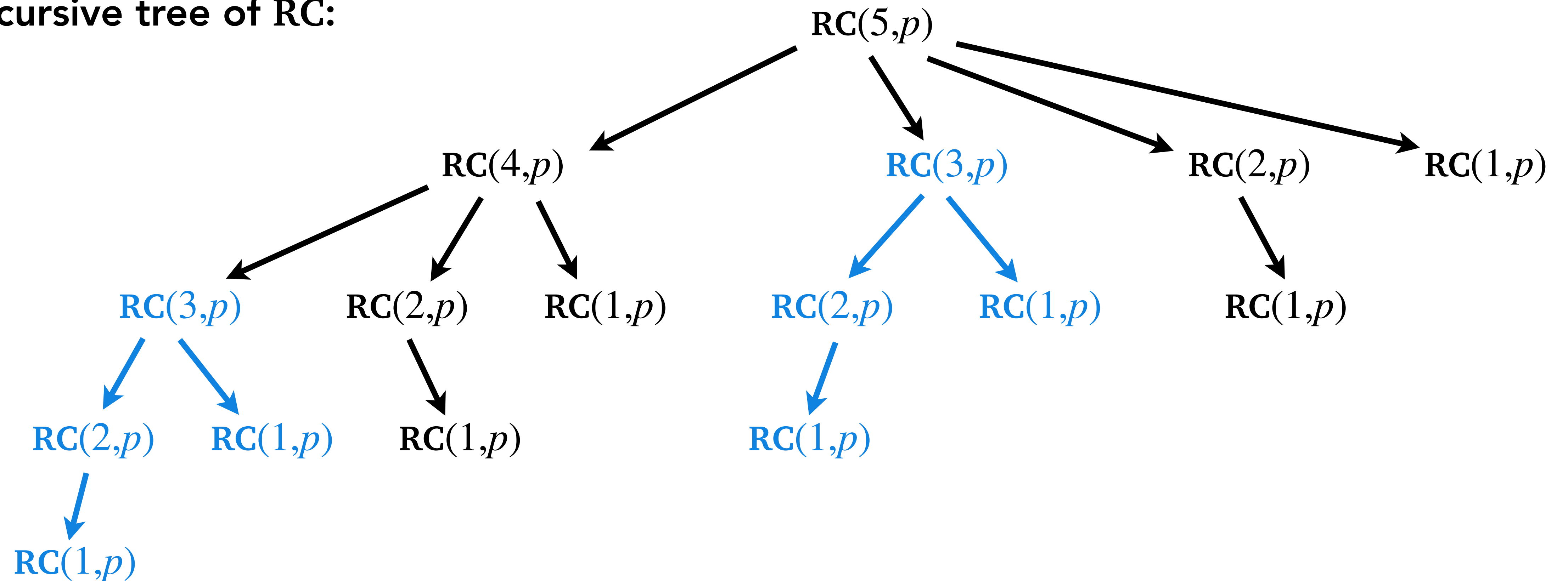
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

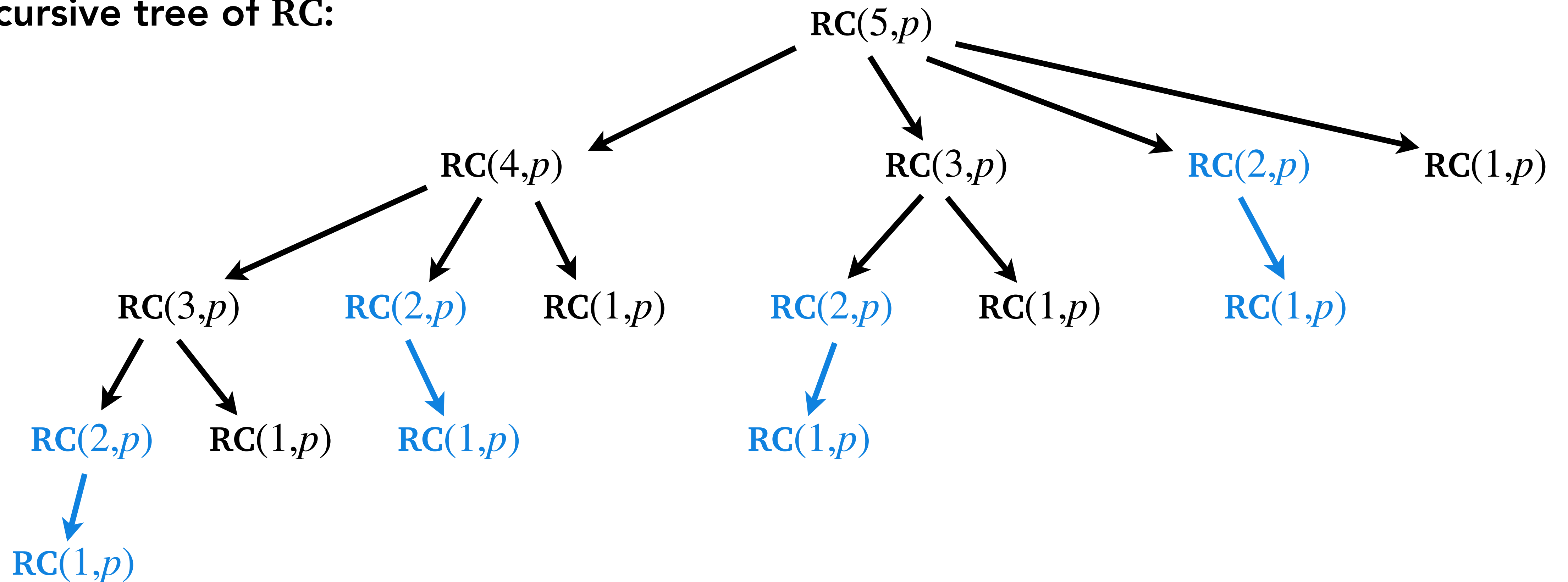
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

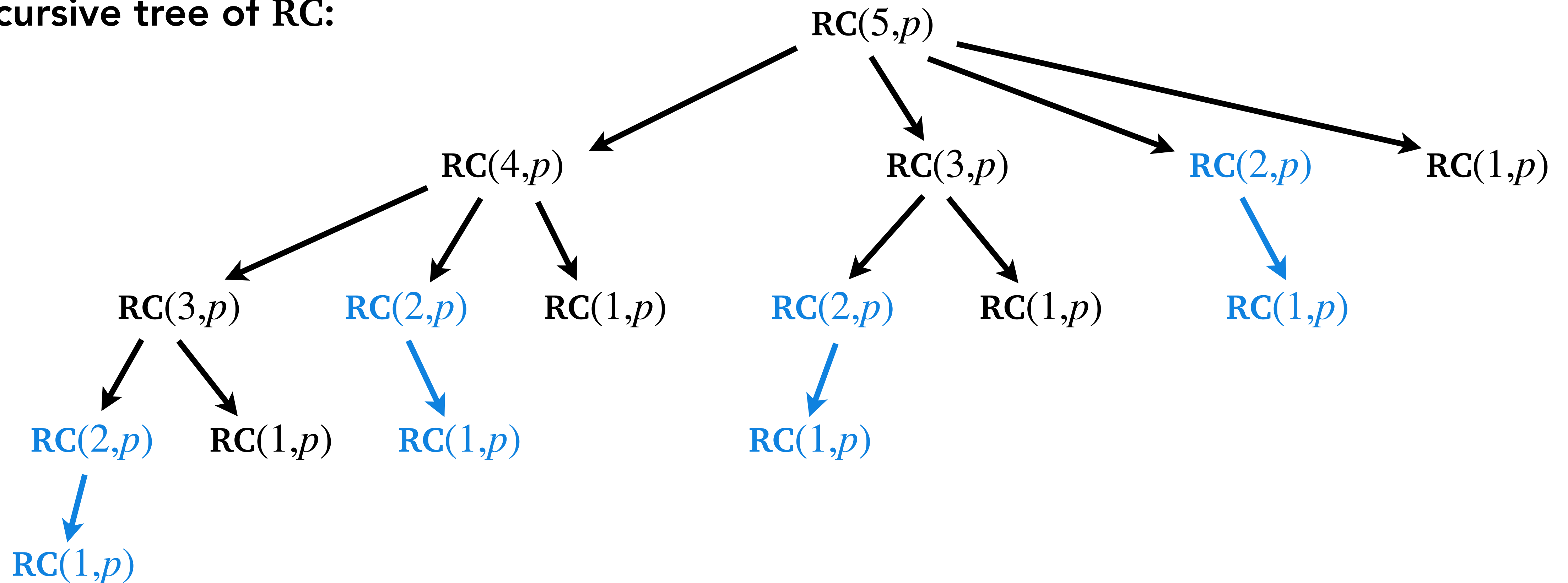
Recursive tree of RC:



All Good?

Even this recursive algorithm **RC** is slow and takes $O(2^n)$ time. (Why $O(2^n)$?)

Recursive tree of RC:



Observation: $\text{RC}(3,p)$ and $\text{RC}(2,p)$ is getting computed from scratch multiple times.

Dynamic Programming Enters

Dynamic Programming Enters

Idea:

Dynamic Programming Enters

Idea: 1) Define an array *profit*[1 : *n*]

Dynamic Programming Enters

Idea: 1) Define an array *profit*[1 : *n*]

2) Store the maximum profit obtainable from an *i* length rod in *profit*[*i*] for future use after calculating it the first time.

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$$profit[1 : n] = \{p[1], -1, \dots, -1\}$$

Dynamic Programming Enters

Idea: 1) Define an array *profit*[1 : *n*]

2) Store the maximum profit obtainable from an *i* length rod in *profit*[*i*] for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(*n*, *p*):

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(n, p):

1. if $profit[n] \neq -1$ *// profit of n length rod is already computed*

Dynamic Programming Enters

Idea: 1) Define an array *profit*[1 : *n*]

2) Store the maximum profit obtainable from an *i* length rod in *profit*[*i*] for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(*n*, *p*):

1. if $profit[n] \neq -1$ *// profit of n length rod is already computed*
2. return *profit*[*n*]

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(n, p):

1. if $profit[n] \neq -1$ *// profit of n length rod is already computed*
2. return $profit[n]$
3. $profit[n] = p[n]$

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(n, p):

1. if $profit[n] \neq -1$ *// profit of n length rod is already computed*
2. return $profit[n]$
3. $profit[n] = p[n]$
4. for $i = 1$ to $n - 1$

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

$RC(n, p)$:

1. **if** $profit[n] \neq -1$ *// profit of n length rod is already computed*
2. **return** $profit[n]$
3. $profit[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $profit[n] = \text{Max}(profit[n], p[i] + RC(n - i, p))$

Dynamic Programming Enters

Idea: 1) Define an array $profit[1 : n]$

2) Store the maximum profit obtainable from an i length rod in $profit[i]$ for future use after calculating it the first time.

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

$RC(n, p)$:

1. if $profit[n] \neq -1$ *// profit of n length rod is already computed*
2. return $profit[n]$
3. $profit[n] = p[n]$
4. for $i = 1$ to $n - 1$
5. $profit[n] = \text{Max}(profit[n], p[i] + RC(n - i, p))$
6. return $profit[n]$

Time Complexity of RC

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

RC(n, p):

1. **if** $profit[n] \neq -1$
2. **return** $profit[n]$
3. $profit[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $profit[n] = \text{Max}(profit[n], p[i] + \text{RC}(n - i, p))$
6. **return** $profit[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\text{RC}(n, p)$

$\text{profit}[1 : n] = \{p[1], -1, \dots, -1\}$

$\text{RC}(n, p)$:

1. **if** $\text{profit}[n] \neq -1$
2. **return** $\text{profit}[n]$
3. $\text{profit}[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\text{profit}[n] = \text{Max}(\text{profit}[n], p[i] + \text{RC}(n - i, p))$
6. **return** $\text{profit}[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

$\mathbf{RC}(n, p)$:

1. **if** $profit[n] \neq -1$
2. **return** $profit[n]$
3. $profit[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $profit[n] = \mathbf{Max}(profit[n], p[i] + \mathbf{RC}(n - i, p))$
6. **return** $profit[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

$\mathbf{RC}(n, p)$:

1. **if** $profit[n] \neq -1$
2. **return** $profit[n]$
3. $profit[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $profit[n] = \mathbf{Max}(profit[n], p[i] + \mathbf{RC}(n - i, p))$
6. **return** $profit[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

$profit[1 : n] = \{p[1], -1, \dots, -1\}$

$\mathbf{RC}(n, p)$:

1. **if** $profit[n] \neq -1$
2. **return** $profit[n]$
3. $profit[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $profit[n] = \mathbf{Max}(profit[n], p[i] + \mathbf{RC}(n - i, p))$
6. **return** $profit[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\text{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = \underbrace{c'}_{\substack{\uparrow \\ \text{Cost of line 1,2,3, \& 6}}} + O(n) + T(n - 1)$$

Cost of line 1,2,3, & 6

$\text{profit}[1 : n] = \{p[1], -1, \dots, -1\}$

$\text{RC}(n, p)$:

1. if $\text{profit}[n] \neq -1$
2. **return** $\text{profit}[n]$
3. $\text{profit}[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\text{profit}[n] = \text{Max}(\text{profit}[n], p[i] + \text{RC}(n - i, p))$
6. **return** $\text{profit}[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\text{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = \underbrace{c'}_{\text{Cost of line 1,2,3, \& 6}} + O(n) + \underbrace{T(n-1)}_{\text{Cost of RC}(n-1, p)}$$

Cost of line 1,2,3, & 6

$\text{profit}[1 : n] = \{p[1], -1, \dots, -1\}$

$\text{RC}(n, p)$:

1. if $\text{profit}[n] \neq -1$
2. **return** $\text{profit}[n]$
3. $\text{profit}[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\text{profit}[n] = \text{Max}(\text{profit}[n], p[i] + \text{RC}(n - i, p))$
6. **return** $\text{profit}[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\text{RC}(n, p)$

Call to $\text{RC}(n - 1, p)$ computes $\text{profit}[1 : (n - 1)]$

Then,

$$T(1) = c$$

$$T(n) = \underbrace{c'}_{\text{Cost of line 1,2,3, \& 6}} + O(n) + \underbrace{T(n - 1)}_{\text{Cost of RC}(n - 1, p)}$$

Cost of line 1,2,3, & 6

$\text{profit}[1 : n] = \{p[1], -1, \dots, -1\}$

$\text{RC}(n, p)$:

1. if $\text{profit}[n] \neq -1$
2. **return** $\text{profit}[n]$
3. $\text{profit}[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\text{profit}[n] = \text{Max}(\text{profit}[n], p[i] + \text{RC}(n - i, p))$
6. **return** $\text{profit}[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\text{RC}(n, p)$

Call to $\text{RC}(n - 1, p)$ computes $\text{profit}[1 : (n - 1)]$

Then,

$$T(1) = c$$

$$T(n) = \underbrace{c'}_{\text{Cost of line 1,2,3, \& 6}} + \underbrace{O(n)}_{\text{Cost of for loop from } i = 2 \text{ to } n - 1 \text{ as } \text{RC}(n - i, p) \text{ will return immediately.}} + \underbrace{T(n - 1)}_{\text{Cost of } \text{RC}(n - 1, p)}$$

Cost of line 1,2,3, & 6

Cost of for loop from $i = 2$ to $n - 1$
as $\text{RC}(n - i, p)$ will return immediately.

$\text{profit}[1 : n] = \{p[1], -1, \dots, -1\}$

$\text{RC}(n, p)$:

1. if $\text{profit}[n] \neq -1$
2. **return** $\text{profit}[n]$
3. $\text{profit}[n] = p[n]$
4. **for** $i = 1$ **to** $n - 1$
5. $\text{profit}[n] = \text{Max}(\text{profit}[n], p[i] + \text{RC}(n - i, p))$
6. **return** $\text{profit}[n]$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

$$= c' + O(n) + c' + O(n - 1) + T(n - 2)$$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

$$= c' + O(n) + c' + O(n - 1) + T(n - 2)$$

$$= c' + O(n) + c' + O(n - 1) + c' + O(n - 2) + T(n - 3)$$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

$$= c' + O(n) + c' + O(n - 1) + T(n - 2)$$

$$= c' + O(n) + c' + O(n - 1) + c' + O(n - 2) + T(n - 3)$$

$$= c' + O(n) + c' + O(n - 1) + c' + O(n - 2) + c' + O(n - 3) + \dots + c$$

Time Complexity of RC

Let $T(n)$ denote the runtime of $\mathbf{RC}(n, p)$

Then,

$$T(1) = c$$

$$T(n) = c' + O(n) + T(n - 1)$$

$$= c' + O(n) + c' + O(n - 1) + T(n - 2)$$

$$= c' + O(n) + c' + O(n - 1) + c' + O(n - 2) + T(n - 3)$$

$$= c' + O(n) + c' + O(n - 1) + c' + O(n - 2) + c' + O(n - 3) + \dots + c$$

$$= O(n^2)$$

When to use dynamic programming

When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

Optimal Substructure: Optimal solution to the problem contains optimal solutions to

When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

Optimal Substructure: Optimal solution to the problem contains optimal solutions to subproblems.

When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

Optimal Substructure: Optimal solution to the problem contains optimal solutions to subproblems.

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

Optimal Substructure: Optimal solution to the problem contains optimal solutions to subproblems.

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

Overlapping Subproblems: Subproblems have common subsubproblems.

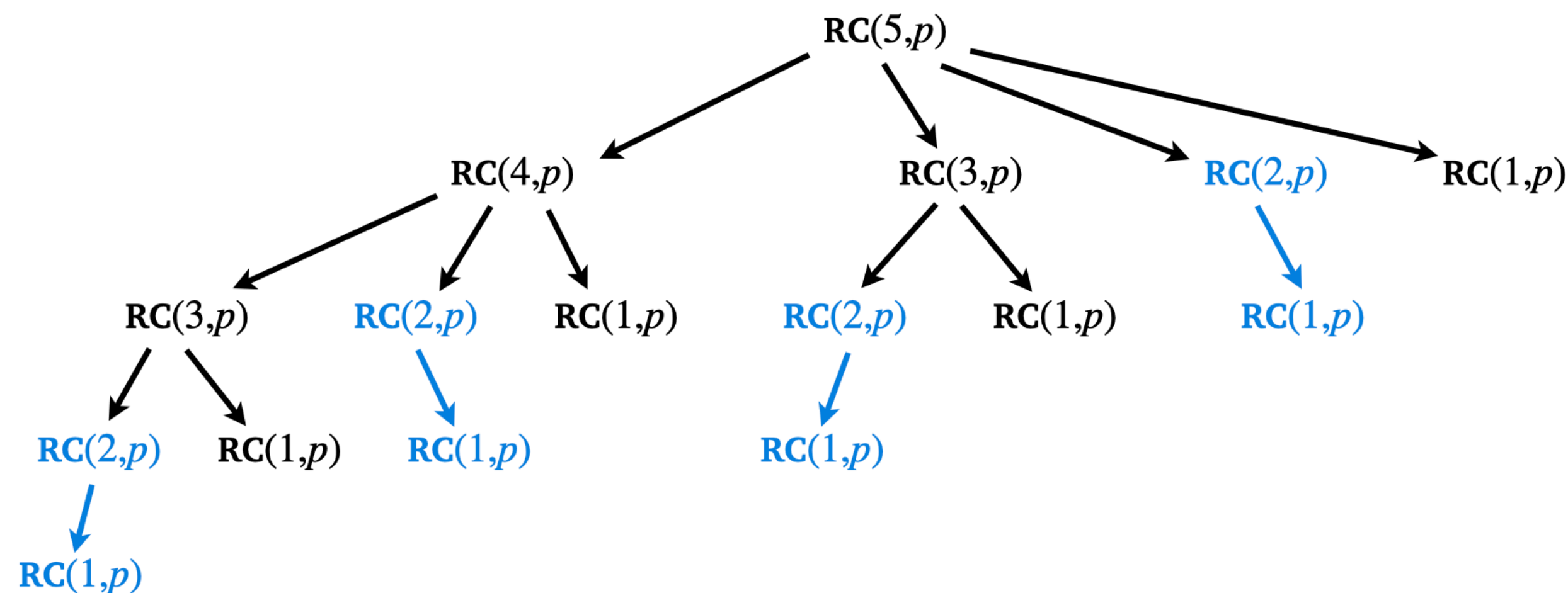
When to use dynamic programming

DP is typically used in **optimization problems** with the following two properties:

Optimal Substructure: Optimal solution to the problem contains optimal solutions to subproblems.

If length of the first cut in optimal cutting is i , then $profit_n = p[i] + profit_{n-i}$

Overlapping Subproblems: Subproblems have common subsubproblems.



How to use dynamic programming

How to use dynamic programming

Solving a problem using dynamic programming usually takes three steps:

How to use dynamic programming

Solving a problem using dynamic programming usually takes three steps:

- Find the **optimal substructure**.

How to use dynamic programming

Solving a problem using dynamic programming usually takes three steps:

- Find the **optimal substructure**.
- **Recursively** define the value of optimal solution.

How to use dynamic programming

Solving a problem using dynamic programming usually takes three steps:

- Find the **optimal substructure**.
- **Recursively** define the value of optimal solution.
- Compute the value of the optimal solution using an **array**.

Bottom-Up Dynamic Programming

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n
3. $profit[j] = p[j]$

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n
3. $profit[j] = p[j]$
4. **for** $i = 1$ **to** $j - 1$

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n
3. $profit[j] = p[j]$
4. **for** $i = 1$ **to** $j - 1$
5. $profit[j] = \text{Max}(profit[j], p[i] + profit[j - i])$

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n
3. $profit[j] = p[j]$
4. **for** $i = 1$ **to** $j - 1$
5. $profit[j] = \text{Max}(profit[j], p[i] + profit[j - i])$
6. **return** $profit[n]$

Bottom-Up Dynamic Programming

Note: 1) Recursive dynamic programming is called **top-down DP**.

2) Iterative DP where we start by solving smaller subproblems “first” is called **bottom-up DP**.

Bottom-Up-RC(n, p):

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. **for** $j = 2$ **to** n
3. $profit[j] = p[j]$
4. **for** $i = 1$ **to** $j - 1$
5. $profit[j] = \text{Max}(profit[j], p[i] + profit[j - i])$
6. **return** $profit[n]$

Time-Complexity: $O(n^2)$ due to loops of line 2 and 4.

Constructing the Optimal Solution

Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

$\text{SolutionRC}(n, p)$

Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$

Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$

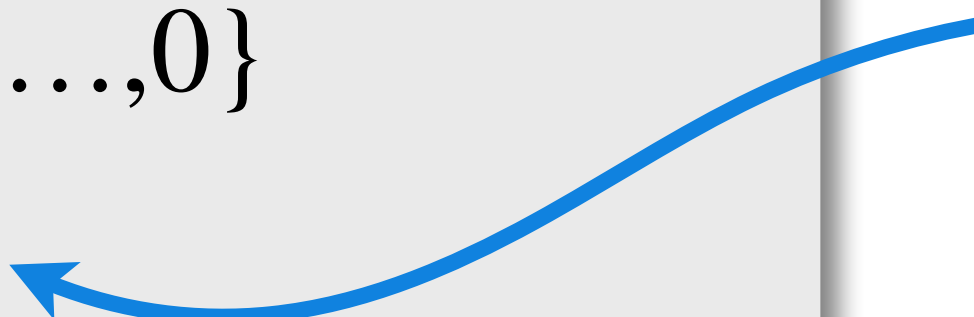
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



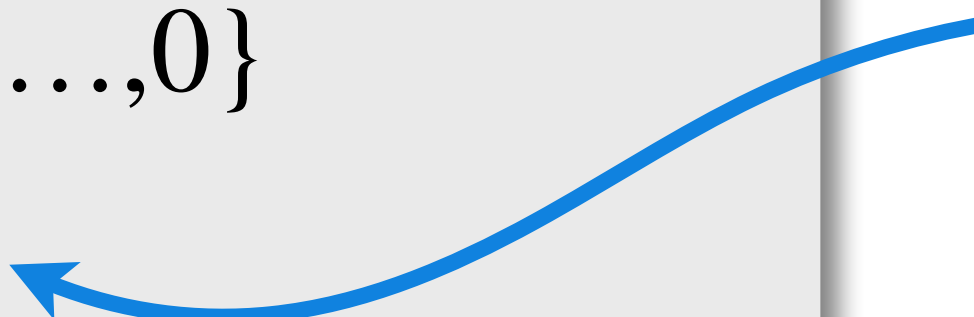
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



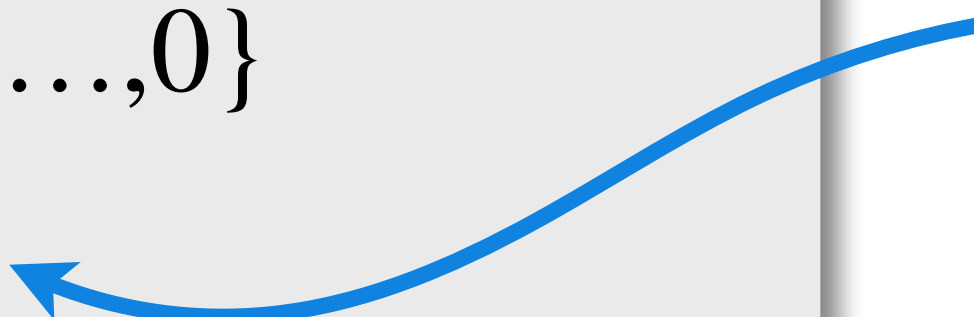
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$

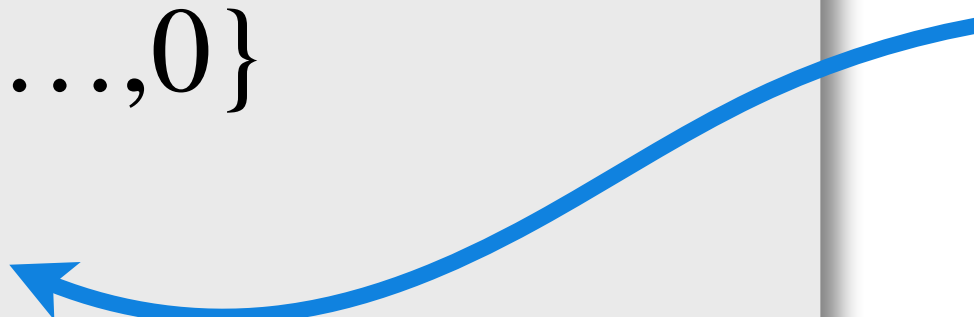
2. $sol[1 : n] = \{1, 0, \dots, 0\}$

3. **for** $j = 2$ **to** n

4. $profit[j] = p[j]$

5. $sol[j] = \dots$

$sol[j]$ is the length of the first cut in
an optimal cutting of j length rod.



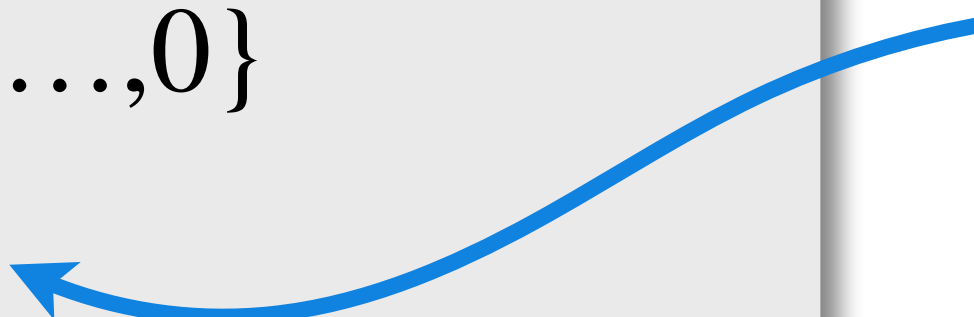
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



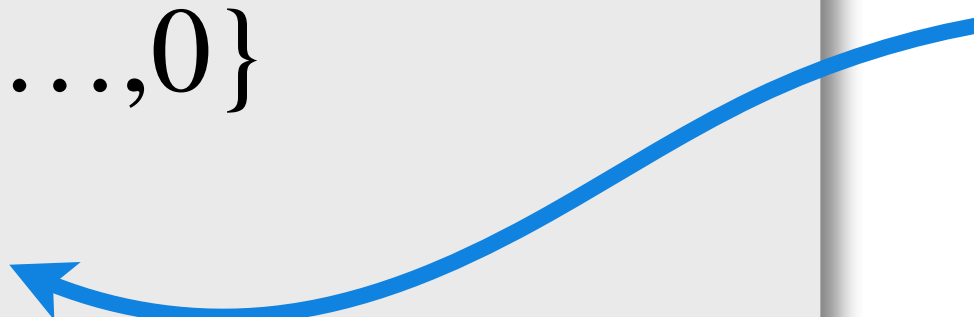
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



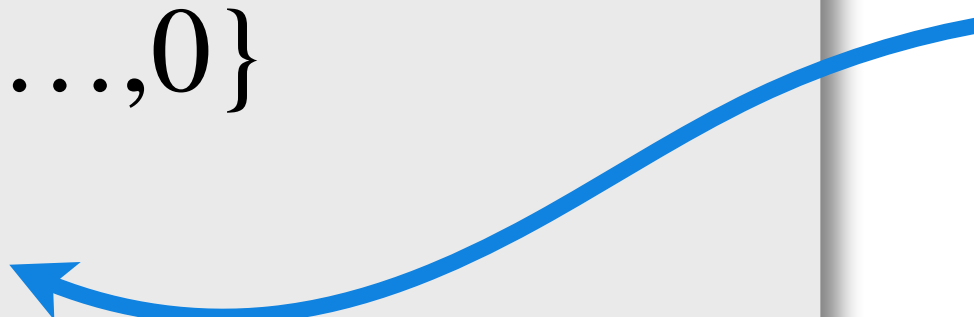
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



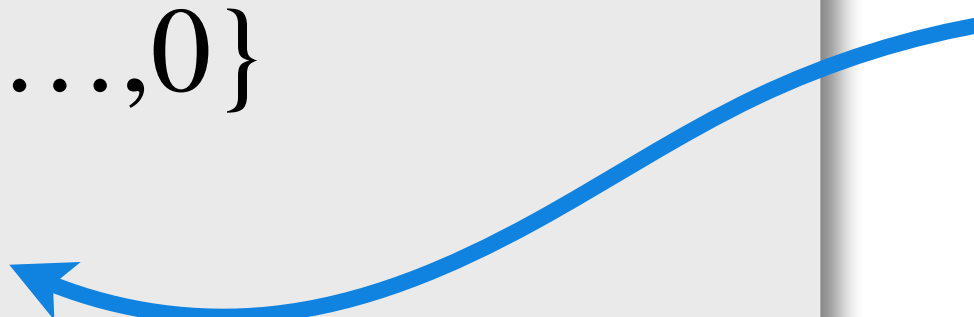
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



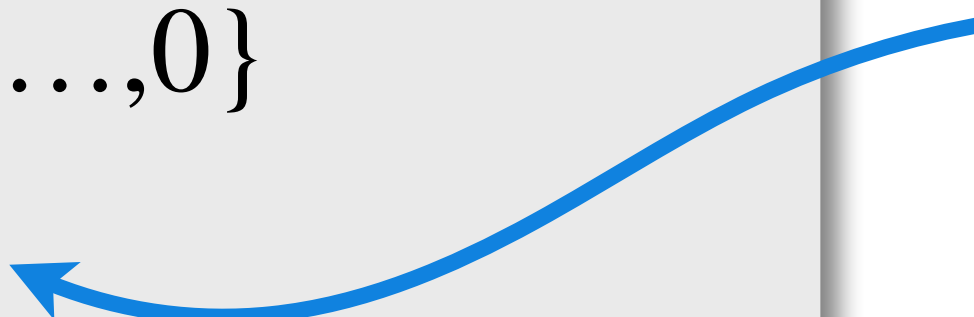
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = \dots$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



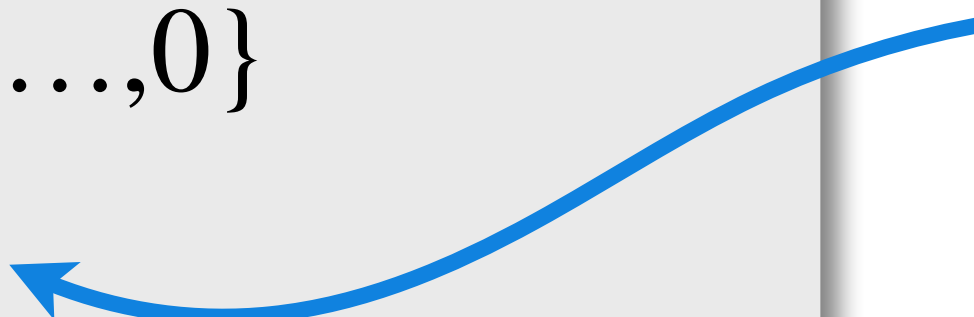
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



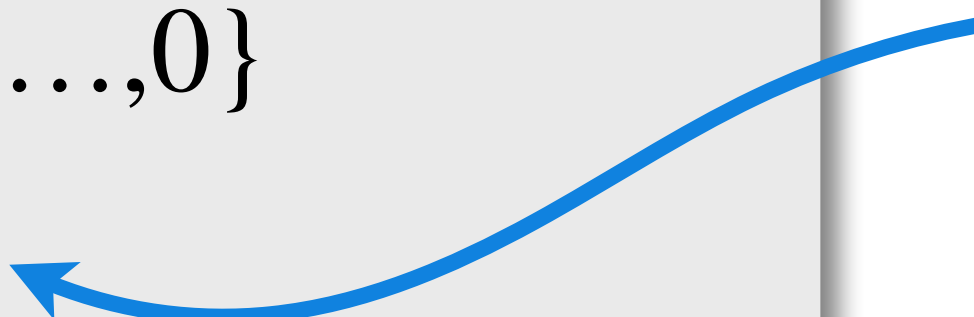
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$
10. **return** $profit[n]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



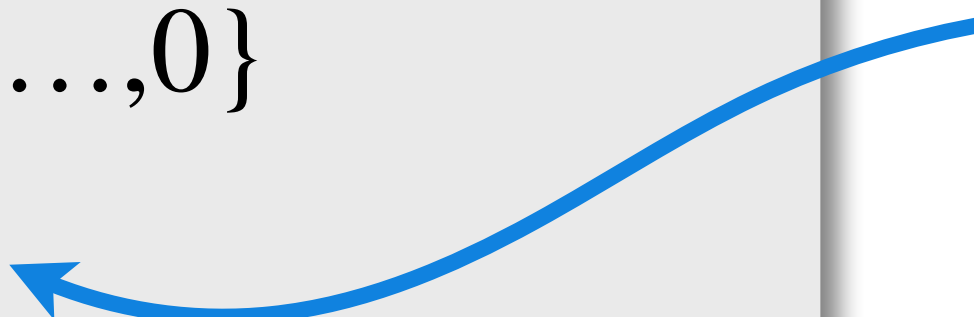
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$
10. **return** $profit[n]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



PrintRC(n, sol)

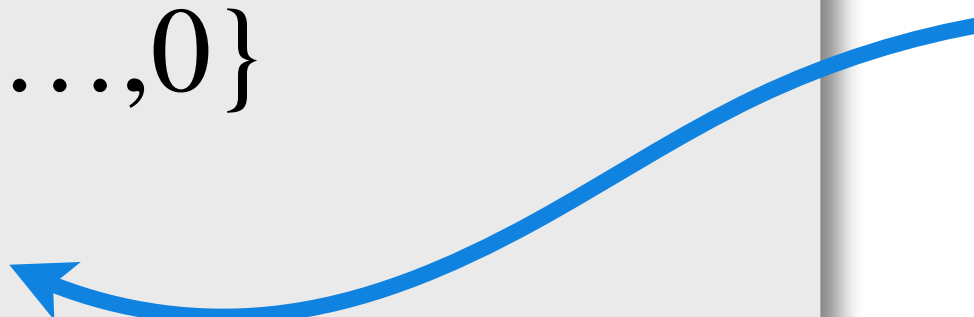
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$
10. **return** $profit[n]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



PrintRC(n, sol)

1. **while** $n > 0$

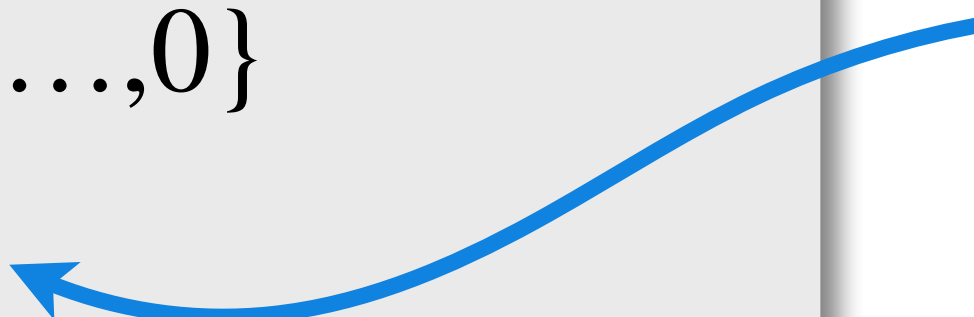
Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$
10. **return** $profit[n]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.



PrintRC(n, sol)

1. **while** $n > 0$
2. **print** $sol[n]$

Constructing the Optimal Solution

Idea: Store the actual choices in a separate array.

SolutionRC(n, p)

1. $profit[1 : n] = \{p[1], 0, \dots, 0\}$
2. $sol[1 : n] = \{1, 0, \dots, 0\}$
3. **for** $j = 2$ **to** n
4. $profit[j] = p[j]$
5. $sol[j] = j$
6. **for** $i = 1$ **to** $j - 1$
7. **if** $profit[j] < p[i] + profit[j - i]$
8. $profit[j] = p[i] + profit[j - i]$
9. $sol[j] = i$
10. **return** $profit[n]$

$sol[j]$ is the length of the first cut in an optimal cutting of j length rod.

PrintRC(n, sol)

1. **while** $n > 0$
2. **print** $sol[n]$
3. $n = n - sol[n]$